Recurrent Neural Networks & Sequential Learning

A Comprehensive Guide from Vanilla RNNs to LSTMs and GRUs for NLP

Facilitator: Nirajan Bekoju

Section 1:
The Challenge of Sequential Data

Deep Learning → ANN, Dense Neural Network → Regression and Classification Tasks.

→ Computer Vision → CNNs

Stock Market Prediction (Nabil Bank)

→ p1, p2, p3, p4, p5 → temporal dependencies
→ I love machine learning →

Order Matters → Sequential Data

Motivation: Why Order Matters

"I love deep learning."

Positive sentiment, specific subject.

"Learning loves me."

Different meaning, same words.

Feed-forward networks treat inputs as independent. But sequences are interdependent.

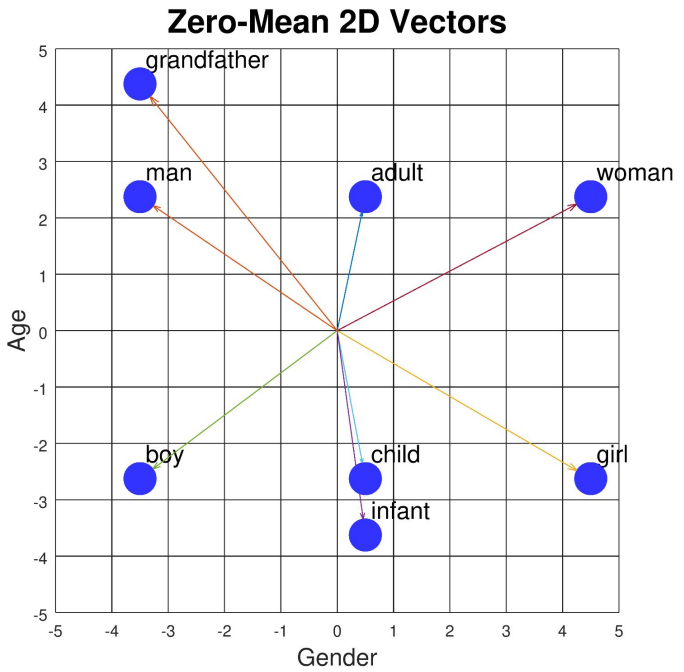
Sequential Data Examples

- **Text:** Words in a sentence.
- **Speech:** Audio waves over time.
- **DNA:** Nucleotide sequences.
- **Stock Prices:** Temporal trends.

Analogy: Watching a movie frame-by-frame vs. looking at a collage of random frames. In the first case, you understand the plot (the context). Standard networks are like the collage—they lack "memory" of what came before.

Limitations of Feed-Forward NNs

- **Fixed Input Size:** FFNNs require vectors of fixed length (e.g., 500 dimensions).
- **Hard Clipping:** Sentences vary in length. Forcing them into fixed buckets leads to information loss.
- **No Temporal Memory:** Each word is processed independently without context from the word before it.



Why CNNs are Not Enough

CNN: Local Receptive Field

Great at spatial features but fixed in size. It looks at a window (kernel) but doesn't persist state across distant samples naturally.

RNN: Sequential Modeling

Designed to process sequences of arbitrary length. It maintains a **Hidden State** that evolves with every input step.

CNNs are spatial experts (images), while RNNs are temporal experts (video, text). While 1D CNNs can handle some sequences, they have a limited horizon. RNNs theoretically have an infinite horizon.

Sequential Processing Intuition

Imagine reading a sentence. You don't start from scratch at every word; you maintain a mental "gist" of the sentence so far.

Example: Human Reading is the perfect analogy. As you read the word 'sat', you still remember it was a 'cat'. The information from "The cat" is summarized in your brain's hidden state.

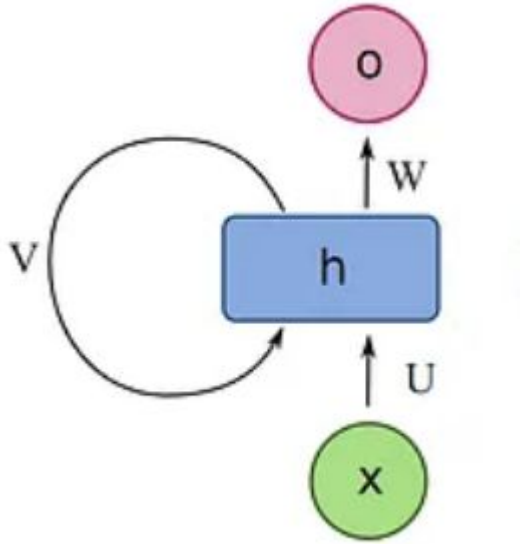
→ Each word updates your understanding (**Hidden State**).

Section 2:
Recurrent Neural Networks

Basic RNN Architecture

An RNN is a network with a **loop**. This loop allows information to persist.

```
Unfolding the loop reveals a chain-like
structure across time steps.
```

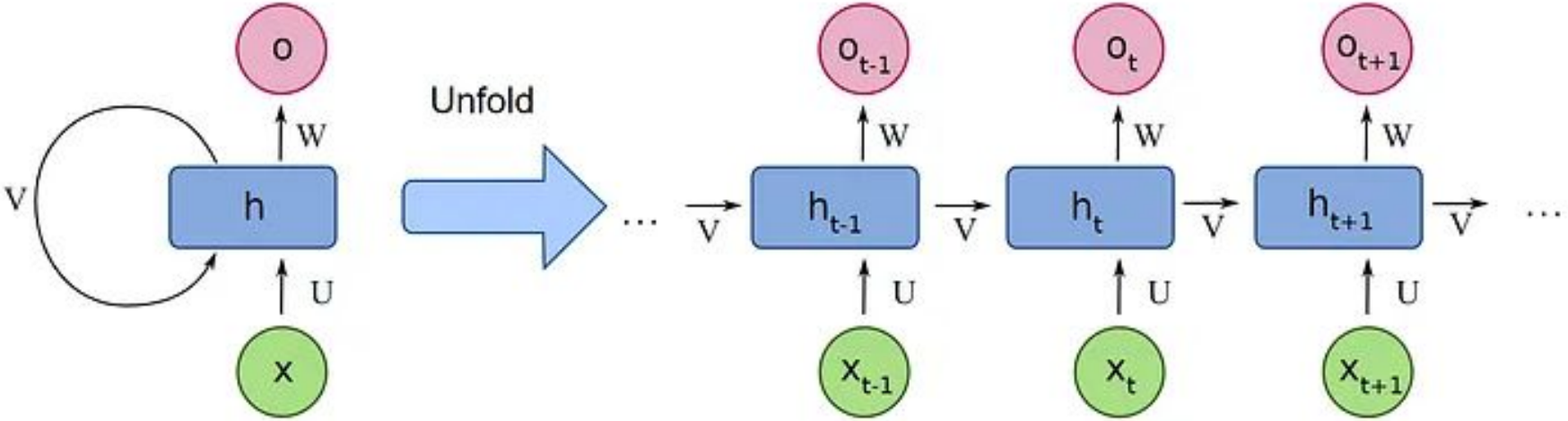


$\mathbf{X_t}$: Input at time t

$\mathbf{h_t}$: Hidden state at time t

In the diagram, a chunk of neural network, A, looks at some input x_t and outputs a value h_t. A loop allows information to be passed from one step of the network to the next.

The "loop" is just a shorthand. In practice, we "unfold" the network for the length of the sequence during training. Note that the weights (W) are SHARED across every time step.



I like potatoes.

$$h_t = \tanh\left(W_{hh} h_{t-1} + W_{xh} x_t + b_h \right)$$

Hidden State: A function of the previous state and current input.

$$y_t = W_{hy} h_t + b_y$$

Output: A linear projection of the hidden state (often followed by Softmax).

Training: BPTT

Backpropagation Through Time

We unfold the network and treat it as a deep feed-forward network with shared weights.

Gradient Flow: Error at step t flows backward to update weights W and propagates to previous time steps.

BPTT is just standard backprop but we have to sum the gradients for the shared weights across all time steps. This becomes computationally expensive for long sequences.

The Vanishing Gradient and Exploding Gradient

As the gradient travels back through time, it is repeatedly multiplied by the weights.

Vanishing gradients occur when gradients become extremely small during backpropagation, causing earlier layers to learn very slowly or stop learning completely.

$$0.5^{20}$$

$$\approx 0.00000095$$

Exploding gradients occur when gradients grow too large during backpropagation, leading to unstable weight updates and divergence in loss. When derivatives or weights are greater than 1, their repeated multiplication across layers leads to exponential growth.

$$1.9^{20}$$

$$\approx 375{,}899.73$$

This is why standard RNNs struggle with long-term memory. The signal simply dies out before it can reach the beginning of the sequence.

The Problem of Long-Term Dependencies

The Clouds are in the..... Sky

Here, Next word prediction “Sky” is easier due to short context.

"The movie, despite being nearly three hours long and having a confusing plot with way too many characters, was actually.... **Fantastic**."

Here, "movie" and "fantastic" are far apart. RNNs might lose the "movie" context.

Real-world NLP is full of these. Pronoun resolution (e.g., "He") might depend on a name mentioned three sentences ago. Standard RNNs fail here.

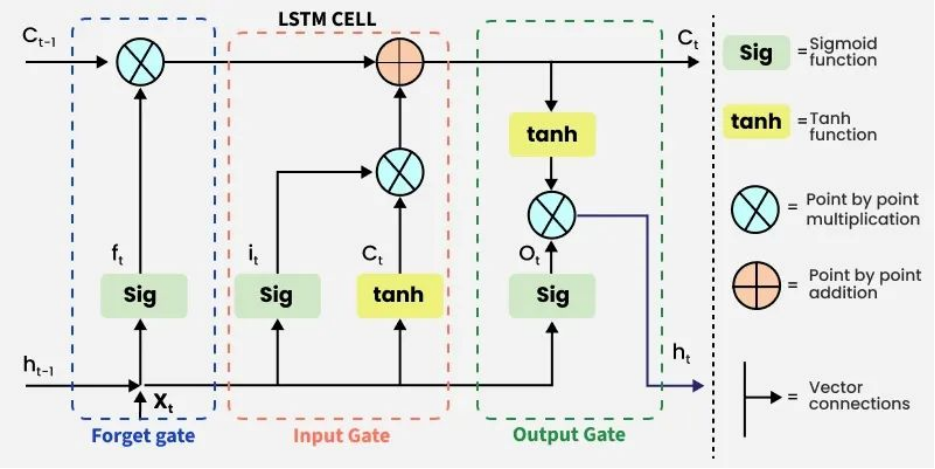
Section 3:
Long Short-Term Memory (LSTMs)

Introducing LSTM

Long Short-Term Memory networks were designed specifically to solve the vanishing gradient problem.

The Core Idea: Gating

Instead of overwriting memory every step, LSTMs use "Gates" to decide exactly what to keep and what to discard.



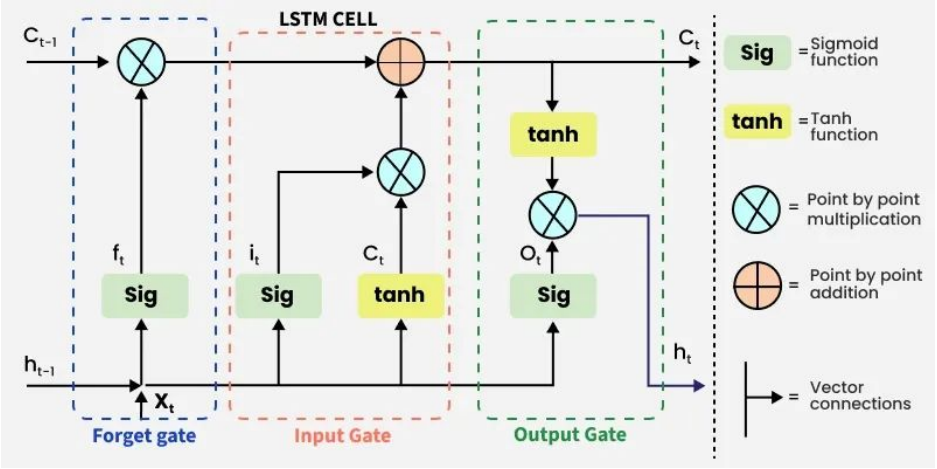
Sigmoid → 0 to 1

Proposed by Hochreiter and Schmidhuber in 1997. It took years to become popular because computers weren't fast enough. Now, it's a staple of modern DL.

LSTM Cell Anatomy

Main Gates in LSTM

- **Forget Gate:** Determines which information should be removed from the memory cell
- **Input Gate:** Decides which new information should be added to the memory cell
- **Output Gate:** Controls which information from the memory cell is passed to the next hidden state and output

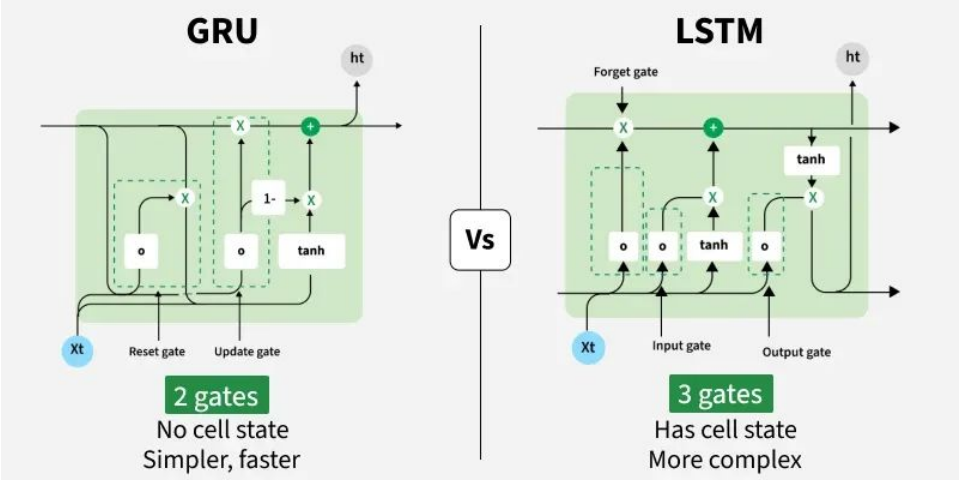


Proposed by Hochreiter and Schmidhuber in 1997. It took years to become popular because computers weren't fast enough. Now, it's a staple of modern DL.

Section 4:
Gated Recurrent Units (GRU)

GRUs: Simpler but Powerful

- Simplified alternative to LSTM.
- Uses update and reset gates to learn long term dependencies



LSTM vs GRU Comparison

Feature	Vanilla RNN	LSTM	GRU
Internal Gates	0	3 (Forget, Input, Output)	2 (Reset, Update)
Memory Component	Hidden State	Cell State + Hidden State	Hidden State
Parameters	Least	Most	Moderate
Training Speed	Fastest	Slowest	Medium-Fast
Performance	Bad performance for long-term sequences	Often better in tasks requiring long-term memory	Performs similarly as LSTMs in many tasks with less complexity

Real-World Applications

Translation

Seq2Seq models for Google Translate style systems.

Speech

Converting audio wave sequences into text strings.

Chatbots

Context-aware conversational AI and generation.

Section 5:
CNNs vs LSTMs

CNN vs RNN for NLP

Different Strengths

CNN (Local): Excels at finding key "keywords" or local patterns (e.g., Sentiment, Spam detection).

RNN (Global): Excels at understanding structural order and long relationships (e.g., Generation, Translation).

Training: CNNs are highly parallelizable (Fast); RNNs are inherently sequential (Slow).

Aspect	CNNs	RNNs
Architecture	Uses convolutional layers to extract features from data.	Utilizes recurrent layers to process sequential data.
Primary Use	Image recognition, computer vision tasks.	Natural language processing, time-series analysis.
Data Input	Fixed-size inputs and outputs.	Variable-size inputs and outputs, sequential data handling.
Memory Handling	No inherent memory of previous inputs.	Maintains a state (memory) of previous inputs.
Parallelization	Highly parallelizable.	Less parallelizable due to sequential processing.